

BALLOON SHOOTING GAME

Assembly Language Project

Project Information

Subject: Assembly Language Programming

Department: Computer Science

Academic Year: 2023 - 2027

Team Members

Nahah Butt | Muhammad Talha

Table of Contents

Table of Contents.....	2
1. Project Overview.....	2
2. Team Members.....	3
3. Objectives	3
4. Game Features	3
4.1 Main Menu Screen	3
4.2 Player Movement.....	3
4.3 Balloon (Projectile Target)	4
4.4 Arrow (Shooting Mechanic).....	4
4.5 Collision Detection	4
4.6 Score System	4
4.7 Game Over Screen.....	4
5. Technical Details.....	4
5.1 Architecture	4
5.2 Key Variables (Data Segment)	5
5.3 Procedures	5
5.4 Game Loop Flow	5
6. How to Run	5
6.1 Requirements	5
6.2 Assembly & Execution Steps	6
6.3 Controls	6
7. Challenges & Solutions	6
8. Conclusion	7
9. References.....	7

1. Project Overview

Balloon Shooting Game is an interactive arcade-style game developed entirely in x86 Assembly Language (8086), targeting real-mode DOS using BIOS and DOS interrupts for all input, output, and screen rendering. The game demonstrates core concepts of low-level programming including direct video memory manipulation, keyboard interrupt handling, real-time game loop design, and score tracking.

The player controls a character on the left side of the screen and must shoot arrows at balloons that float from right to left. The objective is to hit as many balloons as possible before accumulating 9 misses, at which point the game ends and the score is displayed.

2. Team Members

Role	Member 1	Member 2
Name	Nahah Butt	Muhammad Talha
Student ID	23-UON-0338	23-UON-0446
Responsibilities	Game Logic, Collision Detection	UI Rendering, Score System

3. Objectives

The key learning and development objectives of this project are:

- Apply x86 Assembly Language concepts in a complete, working real-world application.
- Understand and utilize BIOS interrupt (INT 10h, INT 16h) and DOS interrupt (INT 21h) services.
- Directly manipulate video memory (segment 0B800h) for real-time screen rendering without OS abstractions.
- Implement a real-time game loop with keyboard input handling and sprite movement.
- Design and implement collision detection logic at the register level.
- Manage application state (score, positions, flags) using memory-mapped variables.
- Practice modular code design using Assembly procedures (PROC/ENDP).

4. Game Features

4.1 Main Menu Screen

On launch, a welcome screen is displayed with the game title, department name, and control instructions. The game begins only after the player presses Enter, ensuring a clean user experience.

4.2 Player Movement

The player character (represented by ASCII character 125) is positioned on the left side of the screen and can move vertically. Up/Down arrow keys update the player's position by directly modifying the video memory buffer address, achieving smooth movement without screen flicker.

4.3 Balloon (Projectile Target)

A balloon (represented by ASCII character 15 with magenta color attribute) starts from the right side of the screen (video memory offset 3860) and moves left at each game loop iteration. A new balloon spawns automatically after each hit or miss.

4.4 Arrow (Shooting Mechanic)

Pressing the Spacebar fires an arrow (ASCII character 26, arrow symbol) from the player's current position. The arrow travels horizontally to the right. Only one arrow can be active at a time, preventing spamming. The arrow disappears after travelling a limited distance.

4.5 Collision Detection

Collision is detected by comparing the arrow's current video memory position (arrow_pos) with the balloon's position (loon_pos). When they match exactly, a beep sound is produced via DOS interrupt (INT 21h, AH=2, DL=7), the hit count is incremented, and a new balloon is launched.

4.6 Score System

A live score is displayed at the top of the screen using the show_score procedure. It shows both 'Hits' (successful shots) and 'Miss' (balloons that crossed the screen). The score buffer (state_buf) is updated in real-time after every hit or miss event.

4.7 Game Over Screen

The game ends when the player accumulates 9 misses. A formatted Game Over screen is displayed with the final score. The player can press Enter to restart the game from the beginning without rerunning the executable.

5. Technical Details

5.1 Architecture

Component	Detail
Processor Target	Intel 8086 / x86 (16-bit Real Mode)
Memory Model	.model Small (64KB Code + 64KB Data)
Assembler	MASM (Microsoft Macro Assembler)
Platform	MS-DOS / DOSBox Emulator
Video Memory Segment	0B800h (CGA Color Text Mode)
Screen Mode	Text Mode 3 (80x25, 16 colors)
Keyboard Input	BIOS INT 16h (AH=0 read, AH=1 status check)
DOS Output	INT 21h (AH=09h string print, AH=02h char print)
Screen Clear	INT 10h (AH=0, AL=3 – reset video mode)

5.2 Key Variables (Data Segment)

All game state is stored in the .data segment using defined variables:

- `player_pos` (DW) — Current video memory offset of the player character.
- `arrow_pos` (DW) — Current video memory offset of the flying arrow.
- `arrow_status` (DB) — Flag: 0 = no arrow active, 1 = arrow in flight.
- `arrow_limit` (DW) — Maximum allowed video offset for the arrow before it disappears.
- `loon_pos` (DW) — Current video memory offset of the balloon.
- `loon_status` (DB) — Balloon active flag.
- `direction` (DB) — Player movement direction: 8 = up, 2 = down, 0 = no movement.
- `hits` (DW) / `miss` (DW) — Cumulative hit and miss counters.
- `state_buf` (DB) — String buffer for score display, printed with INT 21h.

5.3 Procedures

- `show_score` — Updates `state_buf` with current hit/miss values by computing ASCII digits and writes the score string to stdout.
- `clear_screen` — Resets video mode using INT 10h to clear the screen efficiently without iterating over memory cells.

5.4 Game Loop Flow

1. Check if a key is pending in the BIOS keyboard buffer (INT 16h, AH=1).
2. If key pressed: read and dispatch (up/down arrow -> set direction; space -> fire arrow; enter -> restart).
3. Check if miss count ≥ 9 -> jump to `game_over`.
4. Check collision: if `arrow_pos` == `loon_pos` -> hit sequence.
5. Update player position based on direction variable.
6. Check if arrow has exceeded its limit -> hide arrow.
7. Advance balloon position left by 160 (one row in 80-column text mode, 2 bytes per cell).
8. Render balloon, arrow, and player to video memory.
9. Loop back to step 1.

6. How to Run

6.1 Requirements

- DOSBox (recommended) or any MS-DOS compatible environment.
- MASM (Microsoft Macro Assembler) or TASM (Turbo Assembler).
- A Windows / Linux / macOS machine with DOSBox installed.

6.2 Assembly & Execution Steps

10. Install DOSBox and MASM on your system.
11. Place bubbleshootinggame.asm in your MASM working directory.
12. Open DOSBox and mount the directory: MOUNT C C:\MASM
13. Assemble the source: ML bubbleshootinggame.asm
14. Run the executable: bubbleshootinggame.exe
15. Press Enter on the main menu to begin playing.

6.3 Controls

Key	Action
Up Arrow	Move player upward
Down Arrow	Move player downward
Spacebar	Shoot an arrow
Enter	Start game / Restart after Game Over

7. Challenges & Solutions

Challenge	Solution
Real-time rendering without flicker	Write directly to video segment 0B800h instead of using DOS print routines for sprite positions
Collision detection at memory level	Compare 16-bit video offsets of arrow and balloon; when equal, trigger hit event
Single-arrow constraint	Used arrow_status byte flag; spacebar fires only when flag is 0 (no arrow active)
Score display at fixed screen position	Built a pre-formatted string buffer (state_buf) and overwrite it in-place; reprint from same cursor position
Screen clearing between game states	Used INT 10h reset (set video mode) which wipes screen instantly without manual iteration

8. Conclusion

The Balloon Shooting Game successfully demonstrates the power and precision of x86 Assembly Language programming. By building a complete, interactive game from scratch using only low-level instructions, BIOS interrupts, and direct memory access, the team gained deep practical insight into how software interacts directly with hardware.

The project reinforced understanding of the 8086 register model, memory segmentation, interrupt-driven I/O, and real-time application design — all fundamental skills for systems programming, embedded development, and computer architecture. The game is fully functional, replayable, and demonstrates clean code organization through the use of well-defined procedures and labeled code sections.

9. References

- Irvine, K. R. (2014). Assembly Language for x86 Processors (7th ed.). Pearson.
- Intel Corporation. (1979). Intel 8086 Family Users Manual. Intel Corporation.
- Microsoft Corporation. MASM (Microsoft Macro Assembler) Reference Documentation.
- Ralf Brown's Interrupt List — Comprehensive x86 BIOS/DOS interrupt reference.
- DOSBox Project. DOSBox — DOS Emulator. <https://www.dosbox.com>